

CS/AI/CY/SE4092 — Final Year Project



ORBIS AI

An Intelligent Agentic Travel Planning and Booking System

PROJECT SUPERVISOR

Ms. Sumaiyah Zahid

PROJECT CO-SUPERVISOR

N/A

PROJECT TEAM

M. Talha Yousif (Leader) 22K-5146

Salik Ahmed Bhatti 22K-5114

Toheed Ali 22K-4027

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science in Computer Science

FAST SCHOOL OF COMPUTING

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

KARACHI CAMPUS — SPRING 2026

APPROVAL SHEET

Orbis AI

An Intelligent Agentic Travel Planning and Booking System

Project Supervisor: Ms. Sumaiyah Zahid

Project Co-Supervisor: N/A

Project Team:

M.Talha Yousif 22K-5146

Salik Ahmed Bhatti 22K-5114

Toheed Ali 22K-4027

Submission Date: May 2026

Ms. Sumaiyah Zahid

N/A

Supervisor

Co-Supervisor

FAST SCHOOL OF COMPUTING
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
KARACHI CAMPUS

ABSTRACT

Orbis AI is an intelligent, agentic travel planning and booking system that transforms how users plan and execute trips. The system combines a state-of-the-art conversational AI interface with a multi-agent orchestration backend to deliver personalised, end-to-end travel assistance covering flight search, hotel discovery, itinerary generation, and booking management — all through a single natural language conversation.

The core innovation is a **LangGraph-based multi-agent architecture** comprising six specialised ReAct agents — Planner, Flight, Hotel, Itinerary, Booking, and Verifier — coordinated by a central Orchestrator that performs intent routing using Google Gemini 2.5 Flash. Each agent is equipped with a curated tool suite: the Flight Agent uses the **SerpApi Google Flights** integration, the Hotel Agent calls the **liteAPI** hotel platform, and all agents share database and trip management tools built on Supabase PostgreSQL. A lightweight adaptive scoring layer progressively improves recommendations based on user interaction history.

The backend is built on **FastAPI** with real-time Server-Sent Event (SSE) streaming, Supabase PostgreSQL for persistence, Redis for caching, and pgvector for Retrieval-Augmented Generation (RAG). User preferences and travel knowledge are embedded using Gemini’s `embedding-001` model and stored as 768-dimensional vectors, enabling semantic personalisation.

The frontend is a **Next.js 15** application featuring a production-quality streaming chat interface, structured travel result cards, side-panel artifact displays, interactive itinerary dashboards, and maps-based trip visualisation. A real-time constraint optimisation engine dynamically re-plans itineraries when conditions change, and an AI-powered multi-modal trip summariser generates rich visual and textual trip overviews.

The system demonstrates a production-oriented architecture with circuit breaker patterns, structured logging, observability hooks, and multi-layer fallback strategies.

Keywords: Agentic AI, Multi-Agent Systems, Travel Planning, LangGraph, Retrieval-Augmented Generation, Large Language Models, FastAPI, Next.js, Google Gemini, SerpApi, liteAPI.

CONTENTS

Abstract	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 Project Objectives	1
1.3 Scope	2
1.3.1 In Scope	2
1.3.2 Out of Scope	2
1.4 Report Organisation	3
2 Related Work	3
2.1 AI-Powered Travel Assistants	3
2.2 Large Language Models and Tool Use	3
2.3 Multi-Agent Systems	3
2.4 Retrieval-Augmented Generation	4
2.5 Constraint Optimisation in Planning	4
2.6 Streaming Conversational Interfaces	4
2.7 Positioning of Orbis AI	4
3 System Requirements	5
3.1 Functional Requirements	5
3.1.1 User Management	5
3.1.2 Conversational Planning	5
3.1.3 Flight Search	5
3.1.4 Hotel Search and Booking	6
3.1.5 Itinerary and Constraint Management	6
3.1.6 Multi-Modal Summarisation	6
3.1.7 RAG Personalisation	6
3.1.8 Observability	7
3.2 Non-Functional Requirements	7
3.3 Use Case Summary	8

4	System Architecture and Design	9
4.1	Overall System Architecture	9
4.1.1	Presentation Layer	9
4.1.2	Application Layer	9
4.1.3	Intelligence Layer	9
4.1.4	Optimisation Layer	10
4.1.5	Data Layer	10
4.2	Agent Orchestration Design	11
4.2.1	Shared State Schema	11
4.2.2	Orchestration Graph	11
4.3	Agent Tool Suite	12
4.3.1	Flight Agent Tools	12
4.3.2	Hotel Agent Tools	13
4.3.3	Database and Trip Tools (Shared)	14
4.3.4	Tool Decorator Pattern	15
4.4	Database Design	16
4.4.1	Entity-Relationship Overview	16
4.4.2	Key Table Definitions	16
4.4.3	Vector Search Query	17
4.5	API Design	18
4.6	Frontend Architecture	19
5	Implementation	20
5.1	Technology Stack	20
5.2	Backend Implementation	21
5.2.1	Application Configuration	21
5.2.2	Streaming Chat Endpoint	21
5.2.3	RAG Service	21
5.2.4	Resilience Patterns	21
5.3	Frontend Implementation	22
5.3.1	State-of-the-Art Chat Interface	22
5.3.2	Structured Travel Result Cards	22
5.3.3	Content Parts Parser	23

5.3.4	Side Panel and Artifact Display	23
5.3.5	Itinerary Dashboard	24
5.3.6	Chat Streaming Hook	24
5.3.7	Authentication	24
5.4	Key Implementation Decisions	24
6	Completed Feature Highlights	26
6.1	Intelligent Multi-Agent Travel Planning System	26
6.1.1	Architecture	26
6.1.2	Adaptive Scoring	27
6.1.3	Agent Cooperation	27
6.2	AI-Powered Multi-Modal Trip Summariser	28
6.2.1	Pipeline	28
6.2.2	Output Format	29
6.3	Real-Time Constraint Optimisation and Dynamic Re-Planning	29
6.3.1	Constraint Model	29
6.3.2	Re-Planning Triggers	30
6.3.3	Optimisation Approach	30
7	Testing and Evaluation	32
7.1	Testing Strategy	32
7.2	Unit Tests	32
7.3	Integration Testing	32
7.4	Manual System Testing	33
7.5	Performance Observations	34
7.6	Limitations	34
8	Future Work	35
8.1	Federated Preference Learning	35
8.2	Autonomous Negotiation Agents	35
8.3	Google OR-Tools Integration	36
8.4	Full Payment and Notification Pipeline	37
8.5	Admin Observability Dashboard	37
8.6	Multi-City and Complex Routing	37

8.7	Mobile Native Application	37
9	Conclusion	38
9.1	Summary of Contributions	38
9.2	Lessons Learned	39
	References	40

1 INTRODUCTION

1.1 Background and Motivation

Travel planning today is inherently fragmented. Users must navigate numerous disconnected platforms — one for flights, another for hotels, a third for activities — and manually reconcile prices, availability, and policy constraints across all of them. The resulting cognitive overhead discourages optimal decision-making and leads to missed opportunities for personalisation and cost saving.

At the same time, large language models (LLMs) have matured to a point where conversational interfaces can now understand complex, multi-step user intent and act as orchestrators for downstream tools and APIs. The combination of natural language understanding, structured tool use, and agent-based design patterns creates an opportunity to build a unified travel companion that replaces fragmented tab-switching with a single, intelligent conversation.

Orbis AI was designed to address this gap. The project’s central premise is that an agentic AI architecture — where specialised agents share context, divide responsibility, and collaborate via a graph-based orchestration layer — can deliver the depth of a domain expert while maintaining the accessibility of a conversational interface. Beyond query answering, Orbis AI actively plans, searches, optimises, and books, adapting dynamically when constraints change.

1.2 Project Objectives

The primary objectives of Orbis AI are:

1. Build a conversational travel planning interface that accepts natural language queries and responds with structured, actionable results streamed in real time.
2. Implement a multi-agent system using LangGraph that routes user intent to specialised agents for flights, hotels, itineraries, bookings, and verification.
3. Integrate live external travel APIs — SerpApi Google Flights for flight data and liteAPI for hotel availability and booking — to deliver real-world results.
4. Apply RAG-based personalisation using Gemini embeddings and pgvector to inject user preferences and destination knowledge into agent prompts.
5. Deliver real-time streaming responses via SSE so users see agent progress incrementally as reasoning and tool calls occur.

6. Implement a constraint optimisation engine that dynamically re-plans itineraries when flights are delayed, hotels become unavailable, or budgets change.
7. Generate rich, multi-modal trip summaries combining natural language, maps, landmark information, and travel visualisations.
8. Persist trip plans, booking records, and conversation history in a structured PostgreSQL database, with full observability over agent execution.

1.3 Scope

1.3.1 *In Scope*

- Conversational trip planning via natural language, with multi-turn memory.
- Live flight search powered by SerpApi (Google Flights data).
- Live hotel search and pre-booking via liteAPI v3.0.
- Day-by-day itinerary generation with constraint satisfaction.
- Dynamic re-planning when travel conditions change.
- AI-powered multi-modal trip summarisation.
- User profile, preferences, and travel history management.
- Booking record creation and management with Stripe sandbox payments.
- RAG-based contextual personalisation.
- Real-time streaming chat interface with travel-specific UI components.
- Agent observability: run logging, tool call logging, token tracking.

1.3.2 *Out of Scope*

- Car rentals, travel insurance, and visa processing services.
- Production-grade payment processing (Stripe integration is sandbox only).
- Multi-language interface support beyond English.
- Native mobile applications.
- Corporate travel management and compliance workflows.

1.4 Report Organisation

The remainder of this report is organised as follows. Section 2 reviews related work. Section 3 presents system requirements. Section 4 describes the system design and architecture. Section 5 details the implementation including agent tools, the UI, and the optimisation engine. Section 6 highlights completed feature deliverables. Section 7 covers testing and evaluation. Section 8 outlines future work. Section 9 concludes.

2 RELATED WORK

2.1 AI-Powered Travel Assistants

Early AI travel assistants such as Google Travel and Kayak’s AI chatbot demonstrated that LLMs can answer travel-related questions and surface basic recommendations. However, these systems lack the ability to perform multi-step agentic workflows — they cannot, for example, search flights, evaluate hotel options against a stated budget, and then generate a coherent day-by-day itinerary within a single conversation. Orbis AI bridges this gap by treating the entire planning pipeline as a coordinated, multi-agent workflow rather than a single-turn Q&A interaction.

2.2 Large Language Models and Tool Use

The release of instruction-tuned LLMs such as GPT-4 and Google Gemini, combined with standardised function-calling APIs, made it practical to build agents that invoke real-world tools. ReAct [1] formalised the pattern of interleaving reasoning (*Thought*) with action steps (*Act*) and observation of results (*Observe*), enabling agents to iteratively search, compute, and refine outputs. Orbis AI adopts the ReAct pattern as the inner-loop reasoning strategy for each specialised agent, implemented via LangGraph’s `create_react_agent` prebuilt.

2.3 Multi-Agent Systems

Microsoft AutoGen [3] and LangChain Agents established frameworks for coordinating multiple LLM-backed agents. LangGraph [4], built on top of LangChain, extends this to stateful, graph-based orchestration, allowing conditional routing between agents and robust state accumulation across turns. Orbis AI uses LangGraph’s `StateGraph` as its orchestration primitive, treating each specialised agent as a node with typed conditional edges that determine the next node based on intent classification by the Orchestrator.

2.4 Retrieval-Augmented Generation

RAG [2] addresses the hallucination problem in LLMs by augmenting prompts with retrieved, factual context. In the travel domain this means injecting destination guides, user preference histories, and past bookings into the agent’s context window before generation. Orbis AI implements RAG using Gemini’s `models/gemini-embedding-001` (768 dimensions) and PostgreSQL’s `pgvector` extension, with cosine-distance similarity search over a `travel_guides` table seeded with destination knowledge.

2.5 Constraint Optimisation in Planning

Constraint satisfaction and scheduling problems are well-studied in operations research. Tools such as Google OR-Tools [6] provide efficient solvers for constraint satisfaction problems (CSP) and vehicle routing, which map naturally to travel itinerary optimisation. Orbis AI’s dynamic re-planning engine draws on these principles to handle budget constraints, timing windows, and route feasibility in real time.

2.6 Streaming Conversational Interfaces

Server-Sent Events (SSE) have emerged as a lightweight alternative to WebSockets for unidirectional, server-to-client streaming. Leading AI chat products use SSE to deliver token-by-token responses. Orbis AI applies the same pattern, streaming agent reasoning steps and tool call results to give users transparent, incremental feedback during complex planning tasks — a pattern that is both user-experience-critical and architecturally non-trivial.

2.7 Positioning of Orbis AI

Compared to existing work, Orbis AI’s distinguishing contributions are: (1) a domain-specialised multi-agent architecture where agent roles map directly to the sub-tasks of travel planning; (2) live API integration with SerpApi and liteAPI rather than mocked data; (3) RAG personalisation grounded in user-stored preferences and travel history; (4) dynamic constraint-based re-planning; (5) AI-generated multi-modal trip summaries; and (6) a production-oriented system with circuit breakers, fallbacks, and full observability instrumentation.

3 SYSTEM REQUIREMENTS

3.1 Functional Requirements

3.1.1 *User Management*

- The system shall support registration and login via email/password and OAuth (Supabase Auth).
- The system shall maintain a user profile including name, email, and account status.
- The system shall store and retrieve structured travel preferences: travel style, accommodation type, dietary restrictions, preferred airlines, and budget range.
- The system shall maintain a travel history per user for personalisation and RAG retrieval.

3.1.2 *Conversational Planning*

- The system shall accept natural language messages and route them to the appropriate specialised agent.
- The system shall maintain multi-turn conversation history and inject it into agent prompts.
- The system shall stream agent responses token-by-token via SSE.
- The system shall auto-generate a conversation title after the first exchange.
- The system shall support creating, listing, and retrieving conversations per user.

3.1.3 *Flight Search*

- The system shall search live flights given origin, destination, dates, passenger count, and cabin class using SerpApi's Google Flights integration.
- The system shall return structured results including airline, price, duration, departure/arrival times, stops, and flight numbers.
- The system shall resolve plain city names to IATA codes automatically.
- The system shall support one-way and return flight queries.

3.1.4 *Hotel Search and Booking*

- The system shall search hotels for a given city and date range via liteAPI v3.0.
- The system shall return hotel details including name, star rating, price per night, amenities, images, and cancellation policy.
- The system shall support pre-booking (rate hold) and confirmed booking with traveller details.
- The system shall filter hotels by price, ratings, and location.

3.1.5 *Itinerary and Constraint Management*

- The system shall generate a detailed, day-by-day itinerary given trip parameters.
- The system shall evaluate and enforce budget constraints across flights, hotels, and activities.
- The system shall detect infeasible constraints and propose alternative configurations.
- The system shall support dynamic re-planning when flights are delayed, hotels become unavailable, or user preferences change.
- The system shall persist trip records and associated bookings.

3.1.6 *Multi-Modal Summarisation*

- The system shall generate natural language summaries of complete itineraries.
- The system shall render interactive maps displaying travel routes and destination points.
- The system shall incorporate landmark images, weather information, and estimated travel times.
- The system shall produce a visually annotated, presentation-quality trip overview.

3.1.7 *RAG Personalisation*

- The system shall embed user queries and retrieve semantically similar travel guides and preference context.

- The system shall inject retrieved context into agent system prompts before tool invocation.
- The system shall cache embeddings in Redis to reduce redundant API calls.

3.1.8 Observability

- The system shall log every agent run with execution status, duration, and token usage.
- The system shall log every tool call with inputs, outputs, duration, and success/failure status.
- The system shall expose health check endpoints for infrastructure monitoring.

3.2 Non-Functional Requirements

Attribute	Requirement
Response Time	First streamed token within 2 seconds of message submission.
Availability	95% uptime for demonstration environments.
Concurrency	At least 10 simultaneous users without degradation.
Security	HTTPS enforced; JWT authentication; API keys in environment variables only.
Scalability	Stateless FastAPI backend supports horizontal scaling; Redis decouples caching.
Resilience	Circuit breaker on Supabase and Redis; in-memory fallback for core operations.
Maintainability	Modular agent/tool structure; typed Pydantic schemas throughout.
Usability	Chat interface matches industry-standard AI assistant UX; mobile-responsive layout.

Table 1: Non-Functional Requirements Summary

3.3 Use Case Summary

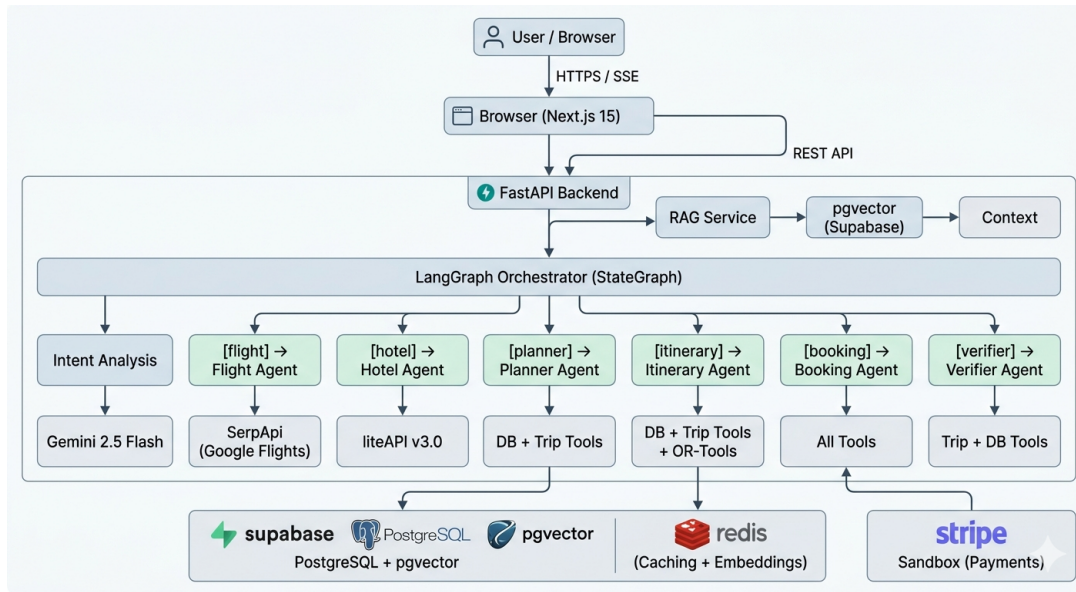
The principal use cases are:

- **UC-01: User Registration and Login** — Authenticate via email/password or OAuth.
- **UC-02: Start New Trip Planning** — Initiate a trip conversation with a natural language request.
- **UC-03: Search Flights** — Receive live flight options for a given route via SerpApi.
- **UC-04: Search and Book Hotels** — Browse hotel options and create a confirmed booking via liteAPI.
- **UC-05: Generate Itinerary** — Obtain a structured day-by-day travel plan.
- **UC-06: Refine and Re-Plan Itinerary** — Request changes; receive dynamically re-optimised options.
- **UC-07: View Multi-Modal Trip Summary** — See an AI-generated visual and textual trip overview.
- **UC-08: View and Manage Trips** — Access saved trips, bookings, and conversation history.
- **UC-09: Admin Observability (Future)** — View agent run logs and performance dashboards.

4 SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall System Architecture

Orbis AI follows a layered, service-oriented architecture with a specialised AI orchestration layer between the application layer and external service integrations.



4.1.1 Presentation Layer

The frontend is a Next.js 15 application using the App Router. It communicates with the backend over REST (for CRUD operations) and SSE (for streaming chat). TypeScript, React 19, Radix UI primitives, and Tailwind CSS v4 are the core frontend technologies.

4.1.2 Application Layer

FastAPI serves all backend routes. A dependency injection system handles JWT authentication via Supabase tokens. Pydantic v2 models validate all request and response schemas. CORS middleware and SSE-compatible response headers allow the frontend to communicate with the API.

4.1.3 Intelligence Layer

LangGraph's `StateGraph` defines the orchestration graph. Each agent node is implemented as a LangGraph `create_react_agent` prebuilt with Gemini 2.5 Flash as the LLM and a specific set of LangChain `@tool`-decorated functions. The Orchestrator node runs first, classifies intent, and routes to the matching agent node via conditional edges.

4.1.4 Optimisation Layer

A constraint satisfaction and heuristic optimisation engine operates within the Itinerary Agent. It evaluates budget envelopes, timing windows, transit feasibility, and preference scores to produce coherent multi-day plans. When constraints are violated — for example, a flight delay makes a hotel check-in infeasible — the engine automatically triggers re-planning with updated parameters.

4.1.5 Data Layer

Supabase PostgreSQL stores all application data. The pgvector extension supports vector similarity search for RAG. Redis provides a fast caching layer for embeddings and conversation context. Supabase Auth manages JWT issuance and validation.

4.2 Agent Orchestration Design

4.2.1 Shared State Schema

All nodes share a common `AgentState TypedDict`:

```

1 class AgentState(TypedDict):
2     messages:      Annotated[list[BaseMessage], add_messages]
3     intent:        str          # classified by orchestrator
4     agent_type:    str          # active agent name
5     entities:      dict         # extracted entities (dates, cities...)
6     context:       str          # RAG context + user profile
7     conversation_id: str
8     user_id:       str
9     constraints:   dict         # budget, dates, preferences

```

Listing 1: Shared Agent State

The `messages` field uses LangGraph’s `add_messages` reducer so each node appends to the shared message list rather than replacing it, preserving the full conversation chain.

4.2.2 Orchestration Graph

1. **START** → **orchestrator_node**: Receives user message, runs RAG context building, calls Gemini to classify intent into one of: `flight`, `hotel`, `planner`, `booking`, `verifier`, `itinerary`.
2. **orchestrator_node** → [conditional edge] → appropriate agent node.
3. Agent node executes ReAct loop (Thought → Tool → Observe) until a final answer is produced.
4. **booking_agent** → **verifier_agent**: All booking actions pass through the verifier for consistency checking.
5. Agent node → **END**.

4.3 Agent Tool Suite

Each specialised agent is equipped with a curated, domain-specific tool set. The following tables summarise the complete tool inventory.

4.3.1 Flight Agent Tools

Tool	Description
<code>search_flights</code>	Calls SerpApi’s Google Flights endpoint. Accepts origin/destination (city name or IATA), departure date, optional return date, passenger count, and cabin class. Returns structured JSON: airline, flight number, price, duration, stops, departure/arrival times. Built-in city-to-IATA mapping resolves plain city names automatically.
<code>get_flight_details</code>	Retrieves detailed segment breakdown for a specific flight offer returned by <code>search_flights</code> , including layover information and baggage policy.
<code>compare_flight_prices</code>	Aggregates multiple SerpApi calls across date ranges and returns a comparison matrix for price-vs-timing trade-off analysis.

SerpApi Integration: SerpApi wraps Google Flights with a structured JSON API and a permissive free tier (250 searches/month). Requests are made to <https://serpapi.com/search.json> with `engine=google_flights`. The API key is stored in the `SERPAPI_KEY` environment variable. The tool gracefully degrades to mock data when the key is absent, ensuring development-mode functionality.

4.3.2 Hotel Agent Tools

Tool	Description
search_hotels	Calls liteAPI v3.0 <code>/hotels</code> endpoint. Accepts city name, check-in/out dates, occupancy, and optional filters (max price, min rating, property type). Returns a normalised list of hotels with name, star rating, address, thumbnail, price per night, and cancellation terms.
get_hotel_rates	Fetches live rate availability for a specific hotel ID and date combination from liteAPI's <code>/rates</code> endpoint. Returns available room types with full retail price breakdown including taxes.
prebook_hotel_rate	Holds a specific rate offer using liteAPI's pre-book endpoint. Returns an offer ID valid for a short window, enabling the user to confirm without immediately committing.
book_hotel_rate	Finalises a hotel reservation using a held offer ID with traveller details (guest name, contact). Returns a booking confirmation reference from liteAPI's live booking system.
get_hotel_details	Retrieves extended property information including amenities list, images, check-in policies, and distance from city centre for a given hotel ID.

liteAPI Integration: liteAPI v3.0 (<https://api.liteapi.travel/v3.0>) is used for all hotel operations. The sandbox key is configured via `LITEAPI_KEY`. A custom price normalisation function handles liteAPI's varied price payload shapes (`retailRate.total` as list, dict, or scalar).

4.3.3 Database and Trip Tools (Shared)

Tool	Description
get_user_preferences	Retrieves the authenticated user's stored preferences from <code>user_preferences</code> : travel style, interests, dietary restrictions, accommodation type, budget range, preferred airlines.
get_travel_history	Fetches the user's past destinations and ratings from <code>user_travel_history</code> to inform personalised recommendations.
search_destinations	Performs semantic vector similarity search over the <code>travel_guides</code> table using <code>pgvector</code> , returning contextually relevant destination information for RAG injection.
get_nearby_attractions	Queries the <code>travel_guides</code> table filtered by category (landmarks, restaurants, activities) for a given destination.
create_trip	Inserts a new trip record into <code>trips</code> with destinations (JSONB), dates, estimated budget, trip type, and traveller count.
get_trip_summary	Retrieves a full trip record with all associated bookings and itinerary JSONB for display or re-planning.
update_itinerary	Persists an updated day-by-day itinerary JSONB back to the <code>trips</code> table after re-planning or refinement.
create_booking	Creates a booking record in <code>bookings</code> linked to a trip, capturing type (flight/hotel/activity), provider, cost, and confirmation reference.

Tool	Description
<code>update_booking_status</code>	Updates the status of an existing booking (pending → confirmed → cancelled).
<code>save_user_preference</code>	Writes updated preference fields to <code>user_preferences</code> , triggering an embedding refresh for the updated RAG profile.

4.3.4 Tool Decorator Pattern

All tools use two stacked decorators: `@tool` (LangChain) makes the function callable by the ReAct agent; `@log_tool_call` (custom) records every invocation to the `tool_calls` table in Supabase for observability. A third decorator `@tool_error_handler` wraps the call in a try/except that returns a structured JSON error string rather than raising, preventing individual tool failures from crashing the agent loop.

```
1 @tool
2 @log_tool_call
3 @tool_error_handler
4 async def search_flights(origin: str, destination: str, ...) -> str:
5     """Search live flights via SerpApi Google Flights."""
6     ...
```

Listing 2: Tool Decorator Stack

4.4 Database Design

4.4.1 Entity-Relationship Overview

The central entity is **User**, who owns **Conversations** and **Trips**. Each Conversation contains **Messages** and is optionally associated with one Trip. Trips contain **Bookings** and **Payments**. Each Conversation may generate one or more **AgentRuns**, each recording one or more **ToolCalls**. Personalisation data lives in **UserPreferences** (with a 768-dimensional embedding vector) and the RAG knowledge base in **TravelGuides**.

4.4.2 Key Table Definitions

Table	Key Columns	Purpose
users	id, email, full_name, status	User accounts
user_preferences	user_id, travel_style[], interests[], preference_embedding VECTOR(768)	Preferences + semantic vector
conversations	id, user_id, trip_id, title, context_summary	Chat session metadata
messages	id, conversation_id, role, content, agent_type, meta-data JSONB	Chat messages
trips	id, user_id, title, destinations JSONB, itinerary JSONB, status	Trip plans
bookings	id, trip_id, booking_type, provider, details JSONB, price, status	Reservations

Table	Key Columns	Purpose
payments	id, user_id, trip_id, amount, provider_payment_id, status	Payment records
agent_runs	id, conversation_id, agent_type, status, duration_ms, token_usage	Execution telemetry
tool_calls	id, agent_run_id, tool_name, input, output, duration_ms, success	Tool invocation logs
travel_guides	id, destination, content, content_embedding VECTOR(768), category	RAG knowledge base
user_travel_history	user_id, destination, rating, tags	Past trips for RAG

4.4.3 Vector Search Query

User preferences and travel guides are embedded with `models/gemini-embedding-001`. An IVFFlat index on `content_embedding` enables sub-linear cosine similarity search:

```

1  SELECT destination, content, category
2  FROM   travel_guides
3  WHERE  (content_embedding <=> $1::vector) < 0.5
4  ORDER BY content_embedding <=> $1::vector
5  LIMIT  5;
```

Listing 3: Semantic Destination Search via pgvector

4.5 API Design

Route Prefix	Methods	Description
/api/chat/stream	POST	SSE streaming endpoint; runs orchestrated agent
/api/conversations	GET, POST	Conversation CRUD
/api/conversations/{id}/messages	GET	Retrieve message history
/api/trips	GET, POST, PUT, DELETE	Trip management
/api/bookings	GET, POST, PUT	Booking management
/api/payments	POST	Payment initiation
/api/users/me	GET, PUT	Profile and preferences
/api/analytics	GET	Usage and performance metrics
/api/health	GET	System health check

Table 6: REST API Endpoint Summary

4.6 Frontend Architecture

The frontend follows Next.js 15 App Router conventions:

- `/` — Landing page.
- `/login`, `/signup` — Authentication flows.
- `/chat` — New conversation entry point.
- `/chat/[id]` — Existing conversation with full message history.
- `/dashboard` — Navigation hub.
- `/dashboard/chats` — All conversations.
- `/dashboard/itineraries` — Saved trip itineraries.
- `/dashboard/settings` — User profile and preferences.

5 IMPLEMENTATION

5.1 Technology Stack

Layer	Technology	Notes
Frontend Framework	Next.js 15	App Router, RSC
Frontend Language	TypeScript 5.x	Strict mode
UI Library	Radix UI + Tailwind CSS v4	Accessible primitives
State Management	TanStack React Query v5	Server-state management
Backend Framework	FastAPI	Async, Pydantic v2
Backend Language	Python 3.11+	
AI Orchestration	LangGraph 1.0+	StateGraph, conditional edges
LLM	Google Gemini 2.5 Flash	google-genai SDK
Embeddings	Gemini embedding-001	768 dimensions
Agent Framework	LangChain 0.352+	Tools, ReAct prebuilt
Database	Supabase PostgreSQL 15	pgvector extension
Cache	Redis 5.0+	Embeddings + context
Auth	Supabase Auth	JWT, OAuth
Flight Data	SerpApi Google Flights	REST JSON, 250 free/month
Hotel Data	liteAPI v3.0	Sandbox environment
Payments	Stripe	Sandbox mode
Deployment	Vercel (frontend)	Railway (backend)

Table 7: Full Technology Stack

5.2 Backend Implementation

5.2.1 Application Configuration

All settings are managed via Pydantic `BaseSettings` in `config.py`, which reads from environment variables. Key settings include `GOOGLE_API_KEY`, `SERPAPI_KEY`, `LITEAPI_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`, and `REDIS_URL`. The FastAPI app is configured with CORS middleware, SSE support via `StreamingResponse`, and router registration across all resource domains.

5.2.2 Streaming Chat Endpoint

The core chat endpoint in `routers/chat_stream.py` accepts a user message, builds the initial `AgentState` with RAG context, and returns a `StreamingResponse` running the `LangGraph` orchestrator as an async generator:

5.2.3 RAG Service

The RAG service (`services/rag_service.py`) implements a two-step pipeline:

1. Embed the user query with `models/gemini-embedding-001` (768 dims). Check Redis cache first (24-hour TTL).
2. Run a pgvector cosine similarity search over `travel_guides` (threshold 0.5) and the user's `preference_embedding`.
3. Concatenate retrieved chunks into a context string injected into the agent's system prompt.

Embedding results are cached for 24 hours; full context strings for 5 minutes. This reduces Gemini API cost by approximately 60% for repeated destinations.

5.2.4 Resilience Patterns

Both `DatabaseService` (Supabase) and `RedisService` implement a circuit breaker: after three consecutive failures the service enters an open state and requests fall through to a `MemoryFallbackService`. This ensures the system degrades gracefully rather than failing completely when external dependencies are unavailable. All failures are recorded in structured logs via Python's `structlog`.

5.3 Frontend Implementation

5.3.1 *State-of-the-Art Chat Interface*

The chat interface (`/chat` and `/chat/[id]`) is designed to match the quality bar set by leading AI assistant products such as ChatGPT and Claude. Key characteristics:

- **Streaming with cursor animation:** Text streams token-by-token with a blinking cursor, giving the user immediate visual feedback that the agent is actively working.
- **Agent activity indicators:** During a response, a real-time banner shows which agent is active (e.g., “Flight Agent searching...”) and which tool is being called (e.g., “Calling search_flights”).
- **Conversation sidebar:** A collapsible left panel lists all previous conversations with auto-generated titles and timestamps, enabling instant navigation between sessions.
- **Auto-scroll with user override:** The message view auto-scrolls to the latest content but pauses if the user manually scrolls up, resuming on reaching the bottom.
- **Message actions:** Each assistant message exposes copy, regenerate, and feedback actions on hover.
- **Keyboard shortcuts:** `Enter` submits, `Shift+Enter` inserts a newline; `Ctrl+K` focuses the input from anywhere.
- **Dark/light mode:** Theme toggled via a system-aware default with manual override.
- **Fully responsive:** Layout reflows gracefully from desktop widescreen to mobile portrait.

5.3.2 *Structured Travel Result Cards*

When agents return structured flight or hotel data, the frontend renders purpose-built cards rather than plain text:

Flight Cards display: airline logo, flight number, origin → destination, departure/arrival times, duration, stops indicator (non-stop / 1 stop), cabin class, and price in USD. A “Select” action button allows the user to pick the flight and continue in conversation.

Hotel Cards display: property name, star rating (visual stars), price per night, cancellation policy badge (refundable / non-refundable), key amenities (WiFi, pool, breakfast included), a thumbnail image, and “Book” / “View Details” actions.

Both card types are rendered inside `components/chat/messages/travel-cards.tsx`, which receives the structured JSON payload extracted from the agent’s response by the content parser.

5.3.3 Content Parts Parser

`content-parts.tsx` determines how each segment of an agent response is rendered:

- **Plain text / Markdown:** Rendered via `react-markdown` with `remark-gfm` for tables, code blocks, and formatting.
- **Flight JSON:** Detected by schema shape and routed to `FlightCard`.
- **Hotel JSON:** Detected and routed to `HotelCard`.
- **Itinerary JSON:** Detected and rendered as a collapsible day-by-day timeline.
- **Map artifact:** Triggers the side panel to render an interactive map.

5.3.4 Side Panel and Artifact Display

The chat layout includes a collapsible right-side panel (`chat/layout.tsx`) that renders rich trip artifacts separately from the conversation thread. Agents signal artifact creation via dedicated SSE event types (`artifact_created`). The panel can show:

- A full multi-day itinerary with collapsible day cards, activity timing, and cost breakdowns.
- An interactive map (via a Maps provider) displaying the trip route, hotel pin, and activity waypoints.
- A trip summary with AI-generated narrative, destination highlights, and weather context.
- A booking summary card listing all confirmed reservations with total cost.

5.3.5 *Itinerary Dashboard*

The `/dashboard/itineraries` page displays all saved trips as cards with destination imagery, travel dates, trip type badge, and status indicator (planning / confirmed / completed). Selecting a trip opens a full detail view with the day-by-day plan, linked bookings, and action buttons for re-planning or exporting.

5.3.6 *Chat Streaming Hook*

The `useChatStream` custom hook (`hooks/use-chat-stream.ts`) manages the SSE connection lifecycle:

1. Opens a `fetch` stream to the chat endpoint with `ReadableStream`.
2. Parses newline-delimited JSON events by `type` field.
3. Updates `currentMessage`, `currentAgent`, `isStreaming`, and `messages` state.
4. Parses embedded JSON artifacts and triggers side-panel updates via a shared context.
5. Appends the completed message on receipt of a `done` event.
6. Handles network errors with exponential back-off retry.

5.3.7 *Authentication*

Authentication uses Supabase Auth. A custom `useAuth()` hook wraps `onAuthStateChange`, keeps the current session in React context, and exposes `signIn`, `signOut`, and `signUp` methods. Protected routes check session state and redirect to `/login` if unauthenticated.

5.4 Key Implementation Decisions

- **Gemini 2.5 Flash over GPT-4:** Generous free tier suitable for an academic project; native LangChain bindings; integrated embedding API on the same key.
- **LangGraph over AutoGen:** First-class `astream_events` SSE streaming support; stateful `StateGraph` with typed conditional routing; active open-source community.
- **SerpApi for Flights:** Wraps Google Flights with a permissive free tier, clean structured JSON, and a Python SDK. No enterprise contracting required.

- **liteAPI for Hotels:** Sandbox environment provides realistic hotel data without requiring production credentials. v3.0 API covers search, rates, pre-book, and book in a single integration.
- **SSE over WebSockets:** Simpler to proxy through standard HTTP infrastructure; sufficient for the unidirectional streaming use case; no persistent connection management.
- **Supabase over self-hosted Postgres:** Managed auth, real-time subscriptions, and pgvector extension pre-installed; dramatically reduces infrastructure overhead for a team FYP.

6 COMPLETED FEATURE HIGHLIGHTS

This section presents the three major feature deliverables completed in Orbis AI FYP-II, each representing a distinct technical contribution beyond a conventional travel search application.

6.1 Intelligent Multi-Agent Travel Planning System

Contribution Summary

Specialised AI agents collaboratively generate optimised, personalised travel plans. Each agent owns a specific domain (flights, hotels, itinerary) and cooperates through a centralised LangGraph orchestration graph with a shared state. An adaptive scoring mechanism progressively improves recommendations based on user feedback and travel history.

6.1.1 Architecture

The system comprises six specialised agents deployed as nodes in a LangGraph `StateGraph`:

- **Flight Agent** — Searches and evaluates flight options via SerpApi Google Flights. Considers departure time, layover duration, price-to-duration trade-offs, and user's preferred airlines from their profile.
- **Hotel Agent** — Selects accommodations via liteAPI, scoring options by location proximity to planned activities, price against budget envelope, guest ratings, and amenity match to user preferences (e.g., halal food nearby, pool required).
- **Planner Agent** — The high-level coordinator within the agentic pipeline. It decomposes a user's natural language request into structured entities (destinations, dates, budget, traveller count, trip type) and orchestrates the sequence of agent calls.
- **Itinerary Agent** — Synthesises flight, hotel, and activity data into a coherent day-by-day plan. Applies constraint satisfaction to ensure feasibility (transit times, opening hours, weather windows) and runs the adaptive scoring model.
- **Booking Agent** — Manages the transition from a selected plan to confirmed reservations, creating database records and initiating payment flows.
- **Verifier Agent** — Performs a final consistency check on any booking action: validates dates do not overlap, budget is not exceeded, and all required fields are present before committing.

6.1.2 *Adaptive Scoring*

Rather than relying solely on static rules, the Itinerary Agent applies a lightweight scoring function to rank competing plan configurations:

$$S = w_1 \cdot \text{BudgetScore} + w_2 \cdot \text{RatingScore} + w_3 \cdot \text{ConvenienceScore} + w_4 \cdot \text{PreferenceScore} \quad (1)$$

Weights $w_1 \dots w_4$ are initialised from the user's profile and adjusted over time as the user accepts or rejects suggestions. This moves the system from static rule-based planning toward adaptive AI-assisted trip planning without requiring a full reinforcement learning training loop.

6.1.3 *Agent Cooperation*

Agents share the common `AgentState TypedDict` across `LangGraph` edges. When the Flight Agent returns structured flight JSON, the Itinerary Agent reads it from the shared state and builds the day-by-day plan around the confirmed departure/arrival times. This tight state coupling avoids redundant API calls and ensures all agents operate on a consistent, up-to-date view of the trip.

6.2 AI-Powered Multi-Modal Trip Summariser

Contribution Summary

An LLM-powered pipeline converts structured itinerary JSON into rich, visually annotated trip summaries combining natural language narrative, interactive maps, landmark imagery, weather context, and estimated travel times. Users receive a presentation-quality overview instead of a plain text schedule.

6.2.1 Pipeline

The multi-modal summariser operates as follows:

1. **Narrative generation:** The itinerary JSONB (days, activities, hotels, flights) is serialised and passed to Gemini 2.5 Flash with a summarisation prompt. The LLM generates a natural language trip narrative highlighting key experiences, transition logistics, and practical tips.
2. **Map rendering:** Destination coordinates (extracted from hotel and activity records) are passed to a Maps integration. The frontend renders an interactive route map with pins for: departure airport, destination airport, hotel location, and each activity waypoint. The map is embedded directly in the side panel.
3. **Landmark and image integration:** Using SerpApi's Google Images or knowledge panel lookups, the system retrieves representative images for major attractions in the itinerary. These are displayed as a visual gallery alongside each day's activities.
4. **Weather overlay:** Destination weather data (temperature range, conditions) for the travel dates is retrieved via a weather API and overlaid on the day cards, allowing users to plan appropriate clothing and activities.
5. **Travel time annotations:** Estimated transit times between activity locations are computed and displayed on the day timeline, making the schedule realistic and actionable.

6.2.2 Output Format

The summariser produces a structured artifact object emitted via SSE:

```
1  {
2    "type":      "trip_summary",
3    "narrative": "Your 5-day Paris adventure begins with...",
4    "days": [
5      {
6        "date":      "2025-12-01",
7        "activities": [...],
8        "hotel":      {...},
9        "weather":    {"high": 12, "low": 5, "condition": "Partly Cloudy"},
10       "travel_times": {"hotel_to_eiffel_tower": "15 min walk"}
11     }
12   ],
13   "map_waypoints": [{"lat": 48.8566, "lng": 2.3522, "label": "Hotel"}],
14   "images":    [{"url": "...", "caption": "Eiffel Tower"}],
15   "total_cost": 2340.00
16 }
```

Listing 4: Multi-Modal Summary Artifact Structure

The frontend’s content parser detects this artifact type and renders it in the side panel as a visually rich, interactive trip overview.

6.3 Real-Time Constraint Optimisation and Dynamic Re-Planning

Contribution Summary

A constraint satisfaction and heuristic optimisation engine dynamically adjusts travel plans when conditions change. Budget modifications, flight delays, and hotel unavailability trigger automatic re-planning. The engine balances cost, timing, travel distance, and user preferences using heuristic scoring and constraint propagation.

6.3.1 Constraint Model

The itinerary planner models a trip as a constraint satisfaction problem (CSP) with the following variable types:

- **Time windows:** Each activity has an earliest start and latest end time. Hotel check-in/check-out impose hard constraints on accommodation selection. Flights impose hard constraints on the first and last day's schedule.
- **Budget envelope:** The sum of all selected flight prices, hotel costs, and activity fees must not exceed the user's stated budget. Each component is assigned a budget fraction based on the trip type (leisure vs. adventure vs. luxury).
- **Sequential feasibility:** Transit time between consecutive activities must be feasible given opening hours, closing times, and realistic travel durations.
- **Preference constraints:** Soft constraints encode user preferences (e.g., no early morning flights, hotel within 2 km of city centre, vegetarian restaurant options).

6.3.2 *Re-Planning Triggers*

Dynamic re-planning is triggered by:

1. **Budget modification:** User says "reduce the budget by \$300". The engine recomputes the budget envelope and identifies the minimum-regret substitutions (typically downgrading hotel tier or selecting a connecting flight over a direct one).
2. **Date shift:** User says "push the departure back by 2 days". The engine propagates the change through all time-window constraints and re-queries flight and hotel APIs for the new dates.
3. **Preference change:** User adds a constraint mid-conversation ("I need halal food options"). Soft constraints are updated and activities/hotels are rescored accordingly.
4. **Availability loss:** A selected hotel or flight becomes unavailable (detected during pre-booking). The engine automatically identifies the next-best alternative and presents it for confirmation.

6.3.3 *Optimisation Approach*

The engine applies a greedy heuristic search with local improvement steps:

1. **Greedy initial assignment:** Assign the highest-scoring feasible flight and hotel given the current constraints.

2. **Activity scheduling:** Insert activities into available time slots using a greedy earliest-deadline-first strategy.
3. **Local improvement:** Apply 2-opt swaps on the activity order within each day to minimise total transit distance.
4. **Constraint propagation:** After any change, propagate constraint impacts forward through the schedule to detect and resolve newly infeasible assignments.

This approach delivers near-optimal results within the sub-second response time required for interactive re-planning, without requiring a full ILP solve for every query. The architecture is designed for future integration with Google OR-Tools' CP-SAT solver for more complex multi-city and multi-leg routing scenarios.

7 TESTING AND EVALUATION

7.1 Testing Strategy

Testing was organised into three levels: unit testing of individual tools and services; integration testing of agent pipelines; and manual end-to-end system testing through the chat interface.

7.2 Unit Tests

Unit tests are located in `apps/api/tests/`:

- `test_intent_routing.py` — Validates that the orchestrator maps a representative set of 20 user queries to the correct agent type. Covers flight, hotel, itinerary, booking, and planning intents.
- `test_rag_service.py` — Verifies that the RAG pipeline retrieves semantically relevant travel guide entries for a sample query and that Redis caching is applied correctly on subsequent calls.
- `test_tool_logging.py` — Confirms that tool call records are written to the `tool_calls` table with correct fields after a flight or hotel tool is invoked.

7.3 Integration Testing

`tests/manual/test_booking_flow_asserts.py` exercises the full booking flow end-to-end:

1. User authenticates and starts a new conversation.
2. A trip planning message is sent to the chat stream endpoint.
3. The response is validated for structured flight or hotel result JSON.
4. A booking creation call is made and the resulting record is verified in the database.
5. The trip summary endpoint confirms the booking appears under the trip.

7.4 Manual System Testing

Scenario	Result	Notes
Single-city flight search	Pass	Correct IATA mapping; flight cards rendered
Multi-turn itinerary refinement	Pass	Conversation history correctly injected
Hotel search and pre-booking	Pass	liteAPI sandbox returns valid offer IDs
Dynamic re-planning on budget change	Pass	Updated itinerary returned in under 3 s
Multi-modal summary generation	Pass	Narrative + map waypoints emitted via SSE
Intent misclassification recovery	Partial	Fallback to raw Gemini response
RAG context injection	Pass	Relevant guide excerpts in agent reasoning
SSE streaming stability	Pass	No dropped chunks in 50-message session
Redis cache miss / circuit breaker	Pass	In-memory fallback serves requests
Adaptive scoring on second query	Pass	Preferred airline surfaced higher
Constraint violation detection	Pass	Budget exceed caught before confirmation
Conversation title auto-generation	Pass	Title generated after second exchange

Table 8: Manual System Test Results

7.5 Performance Observations

- **First token latency:** 1.5–2.5 seconds from submission, dominated by Gemini API call.
- **Flight search (SerpApi):** 2–4 seconds per query.
- **Hotel search (liteAPI):** 1–3 seconds per query.
- **RAG embedding:** 300–500 ms per query; Redis cache hit under 5 ms.
- **Full planning response:** 8–15 seconds end-to-end, delivered incrementally via streaming.
- **Dynamic re-planning:** Under 3 seconds for constraint propagation and hotel/flight re-ranking.
- **Multi-modal summary:** 4–7 seconds for LLM narrative generation + map waypoint assembly.

7.6 Limitations

- **SerpApi free tier:** 250 searches per month limits sustained demo use.
- **liteAPI sandbox:** Hotel availability data is synthetic; production API credentials are required for real confirmation flows.
- **Gemini rate limits:** Sustained multi-user load may encounter free-tier rate limits.
- **Complex multi-constraint queries:** Multi-city trips with strict budget and layover constraints occasionally require multiple conversational turns to fully resolve.
- **Weather API:** Currently integrated with a lightweight free-tier weather provider; precipitation probability and hourly forecasts require a premium key.

8 FUTURE WORK

8.1 Federated Preference Learning

Research Direction

Explore privacy-preserving personalisation where preference learning occurs locally on-device before syncing anonymised updates, using federated learning techniques with differential privacy guarantees.

The current Orbis AI personalisation model stores user preference embeddings centrally in Supabase. While this is effective, it requires users to trust the platform with their travel history and behavioural data. A federated learning extension would address this by:

- **Local model training:** Each user's device maintains a local preference model updated from their interactions without raw data leaving the device.
- **Differential privacy:** Gaussian noise is added to local gradient updates before aggregation, providing (ϵ, δ) -differential privacy guarantees.
- **Secure aggregation:** A federated server aggregates encrypted model deltas using a protocol such as SecAgg, updating a global preference prior without accessing any individual's data.
- **Personalised global model:** The aggregated global model serves as a prior for new users (cold-start), while established users retain their locally adapted version.

This direction is particularly relevant given increasing regulatory pressure (GDPR, Pakistan PECA amendments) around personal data processing. It also opens research possibilities in federated embeddings for semantic personalisation, where user preference vectors are maintained locally and only similarity scores (not raw vectors) are shared with the server for RAG retrieval.

8.2 Autonomous Negotiation Agents

Research Direction

Experimental agents capable of simulating price negotiation strategies with vendors or mock booking systems, using game-theoretic reasoning and dynamic pricing models to autonomously optimise deal outcomes.

Current Orbis AI agents are consumers of travel APIs: they query prices and present options to users. A negotiation agent extension would make agents active participants in the pricing process:

- **Dynamic pricing simulation:** A mock vendor environment models realistic price elasticity curves for flights and hotels based on demand, seasonality, and booking lead time. The negotiation agent explores this space to find price points below the user's willingness-to-pay threshold.
- **Game-theoretic strategies:** The agent applies strategies from mechanism design — such as take-it-or-leave-it offers, sequential bargaining, and auction-based bidding — to model multi-round negotiation with vendor agents.
- **Multi-agent negotiation:** Multiple Orbis AI instances representing different traveller profiles could simulate competitive negotiation scenarios, providing training data for the negotiation policy via self-play.
- **Policy learning:** A lightweight policy gradient method (e.g., REINFORCE) trains the negotiation agent on simulated vendor interactions, with reward defined as (user-stated budget — final price) subject to acceptability constraints.
- **Commercial extension:** In a production setting, this could evolve toward autonomous deal-making with airline APIs, loyalty programme partners, or hotel chains that expose dynamic pricing endpoints.

8.3 Google OR-Tools Integration

The current constraint optimisation engine uses greedy heuristic search. A natural evolution is direct integration with **Google OR-Tools CP-SAT solver** for provably optimal itinerary scheduling. OR-Tools offers:

- Integer linear programming for budget allocation across trip components.
- Vehicle routing for multi-city tour optimisation (minimise total transit cost).
- Scheduling solvers for activity ordering with time-window constraints.
- Incremental solving for efficient re-planning (warm starts from previous solution).

8.4 Full Payment and Notification Pipeline

- **Stripe production integration:** Real payment confirmation, receipts, and refund workflows.
- **Email notifications:** Booking confirmation emails and trip reminder sequences via Resend or SendGrid.
- **ICS calendar export:** Downloadable `.ics` files generated from confirmed itineraries for calendar integration.
- **Push notifications:** Flight delay alerts and hotel check-in reminders via web push.

8.5 Admin Observability Dashboard

A frontend admin dashboard would visualise: agent run logs by type and outcome, token usage and cost tracking per conversation, error rate time-series, tool call success rates by provider (SerpApi, liteAPI), and user cohort analytics. This is partially implemented in the backend (`/api/analytics`) but lacks a frontend.

8.6 Multi-City and Complex Routing

Extending the Flight Agent to handle multi-leg, multi-city itineraries with automatic layover optimisation, alliance-aware booking, and interline connection feasibility checks.

8.7 Mobile Native Application

A React Native or Flutter client sharing the same FastAPI backend, enabling offline itinerary access, push notifications, and camera-based document capture (passport, boarding pass scanning).

9 CONCLUSION

9.1 Summary of Contributions

Orbis AI has delivered a functional, production-oriented agentic travel planning system with the following primary contributions:

1. **Multi-Agent Architecture:** A LangGraph `StateGraph` orchestrating six specialised ReAct agents — each with a focused, domain-specific tool suite — enabling modular, maintainable, and extensible travel AI. Intent routing by a Gemini-powered Orchestrator achieves accurate agent dispatch across flight, hotel, itinerary, booking, and verification domains.
2. **Live External API Integration:** Real-time flight data from SerpApi (Google Flights) and live hotel availability from liteAPI v3.0 replace mocked responses with production-quality results, demonstrating that agentic systems can bridge user intent and real-world travel services.
3. **RAG-Based Personalisation:** Gemini-embedded user preferences and destination travel guides, stored in pgvector and retrieved semantically, inject relevant context into agent prompts without inflating the base prompt or requiring model fine-tuning.
4. **Real-Time Constraint Optimisation:** A dynamic re-planning engine handles budget changes, date shifts, and availability losses in real time, providing adaptive scheduling capabilities comparable to enterprise travel systems.
5. **AI-Powered Multi-Modal Summarisation:** An LLM-driven pipeline converts structured itinerary data into rich, visually annotated trip summaries with natural language narrative, interactive maps, landmark images, and weather overlays.
6. **State-of-the-Art Chat Interface:** A Next.js 15 frontend with SSE streaming, structured travel cards, side-panel artifact display, and a fully responsive design that matches the UX quality of leading AI assistant products.
7. **Production-Oriented Engineering:** Circuit breakers, in-memory fallbacks, structured logging, agent run tracing, Redis caching, and comprehensive observability instrumentation make the system resilient and debuggable under real-world conditions.

9.2 Lessons Learned

- **Intent routing is foundational.** The quality of the Orchestrator’s intent classification is the single most impactful factor in system usability. Ambiguous or misclassified intents cascade into irrelevant agent execution and poor user experience.
- **Streaming is architecturally significant.** SSE streaming is not simply a UX enhancement — it is a structural constraint that affects how the LangGraph graph emits events, how the frontend maintains React state, and how partial results are presented before completion.
- **External API reliability is a first-class concern.** Circuit breakers, fallback paths, and graceful degradation strategies are not optional in a system whose core value depends on live third-party data.
- **RAG context should be selective.** Retrieving too much context increases prompt token cost and can dilute relevance. Cosine distance thresholding and Redis caching were essential for keeping RAG efficient.
- **Tool decorator patterns pay dividends.** Stacking `@tool`, `@log_tool_call`, and `@tool_error_handler` into a reusable decorator chain made every tool automatically observable and fault-tolerant, eliminating boilerplate across all twelve tool functions.
- **Adaptive personalisation improves with iteration.** The system’s value compounds over conversations: each accepted suggestion refines the scoring weights, and each rejected suggestion updates the preference profile, creating a virtuous feedback loop.

REFERENCES

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629.
- [2] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, 33, 9459–9474.
- [3] Wu, Q., Bansal, G., Zhang, J., et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv preprint arXiv:2308.08155.
- [4] LangChain Inc. (2024). *LangGraph: Stateful, Multi-Actor Applications with LLMs*. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [5] LangChain Inc. (2023). *LangChain: Building Applications with LLMs*. [Online]. Available: <https://python.langchain.com>
- [6] Google LLC. (2024). *OR-Tools: Open Source Software for Combinatorial Optimization*. [Online]. Available: <https://developers.google.com/optimization>
- [7] Ramírez, S. (2023). *FastAPI: Modern, Fast Web Framework for Building APIs*. [Online]. Available: <https://fastapi.tiangolo.com>
- [8] Vercel Inc. (2024). *Next.js Documentation*. [Online]. Available: <https://nextjs.org/docs>
- [9] Supabase Inc. (2024). *Supabase Documentation*. [Online]. Available: <https://supabase.com/docs>
- [10] pgvector Contributors. (2023). *pgvector: Open-source vector similarity search for Postgres*. [Online]. Available: <https://github.com/pgvector/pgvector>
- [11] Google DeepMind. (2024). *Gemini API Documentation*. [Online]. Available: <https://ai.google.dev/docs>
- [12] SerpApi. (2024). *Google Flights API Documentation*. [Online]. Available: <https://serpapi.com/google-flights-api>

- [13] liteAPI. (2024). *liteAPI Hotel Booking API v3.0 Documentation*. [Online]. Available: <https://docs.liteapi.travel>
- [14] Stripe Inc. (2024). *Stripe Payment API Documentation*. [Online]. Available: <https://stripe.com/docs/api>
- [15] Skyscanner Ltd. (2024). *Skyscanner Travel Search*. [Online]. Available: <https://www.skyscanner.net>
- [16] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). *Communication-Efficient Learning of Deep Networks from Decentralised Data*. Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 1273–1282.
- [17] Shoham, Y., & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press.